

Laboratory – memory variables

Goal: Understand the memory representation of program variables and constants.

Theoretical aspects

Definitions

Data type represents the set of values together with the operations allowed to be executed upon the values.

Variables are program entities stored by the main memory whose values can be changed during the program execution.

Constants are entities that cannot change their value during program execution.

Pointers are special memory variables whose values are memory addresses where the variable is stored

Characteristics

Language specific data types are based on primary data types supported by the microprocessor.

Characteristics of primary data types have impact on variable usage and program execution. They are not fully transparent for high level programming constructions (e.g. values set supported by a data type).

Characteristics of primary data types:

- Size – number of bytes required to store a value of the type
- Encoding – binary representation of the values set of the data type (Signed vs. unsigned, Integer vs. floating point)
- Operations – instructions which have operands the values of the type.

Memory size of a variable will give the range of values the value can keep

- unsigned int – $[0, 2^{32}-1]$
- signed int – $[-2^{31}, 2^{31}-1]$

Encoding of data – single precision floating point

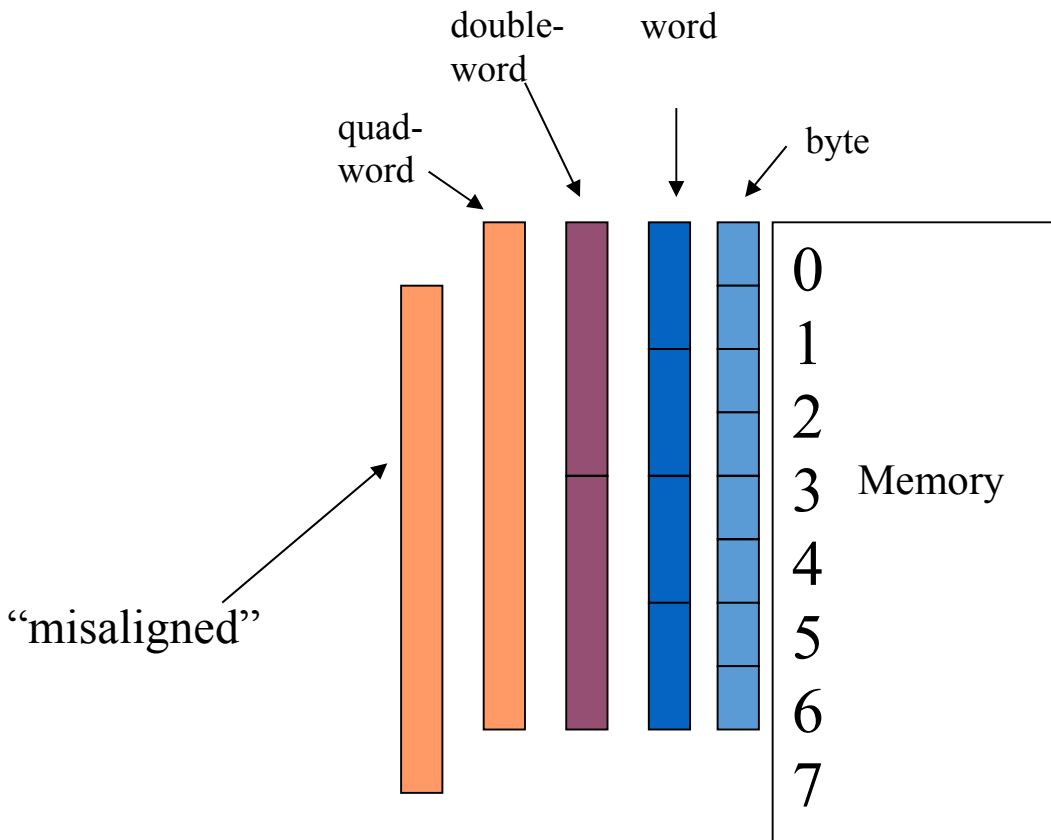
- Standard: IEEE 754
- Size: 32 bits
 - o 1 sign bit (s)
 - o 8 exponent bits (e), biased by 127
 - o 23 mantissa bits (m), prefixed by 1, omitted
- Number: $s \cdot m \cdot 2^e$
 - o $(1-2 \cdot s) \cdot (1+m) \cdot 2^{(e-\text{bias})}$

A variable is characterized by

- Name (identifier) – associated to the memory address of the locations (offset of specific program section)

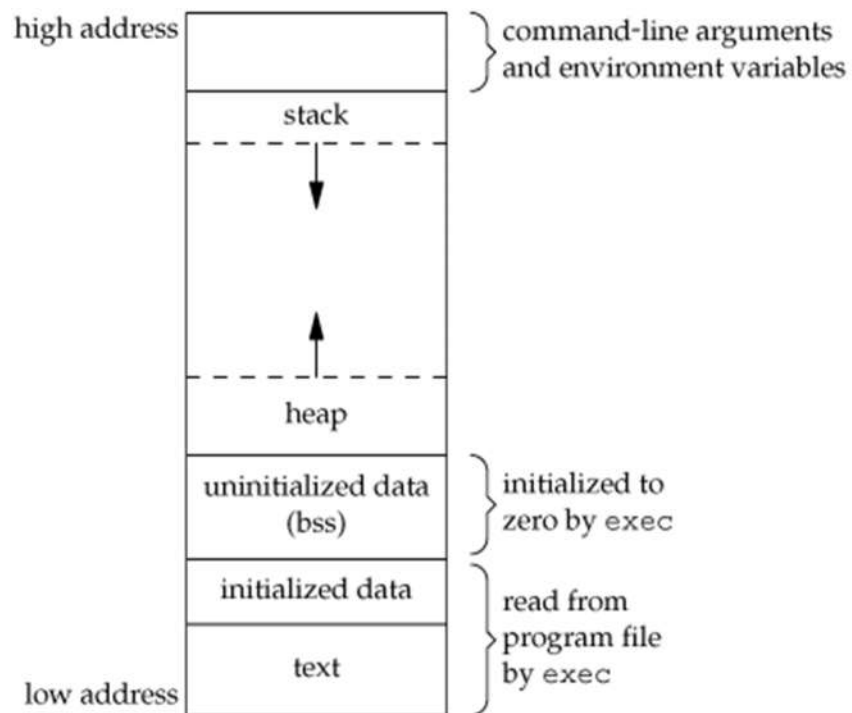
- Type – specifies the set of values the variable can take, their encoding, and the operations allowed to be executed upon the variable
- Value – the information stored by memory locations

Memory address alignment – the variable is placed in memory at an address multiple of the size of variable.



Memory sections

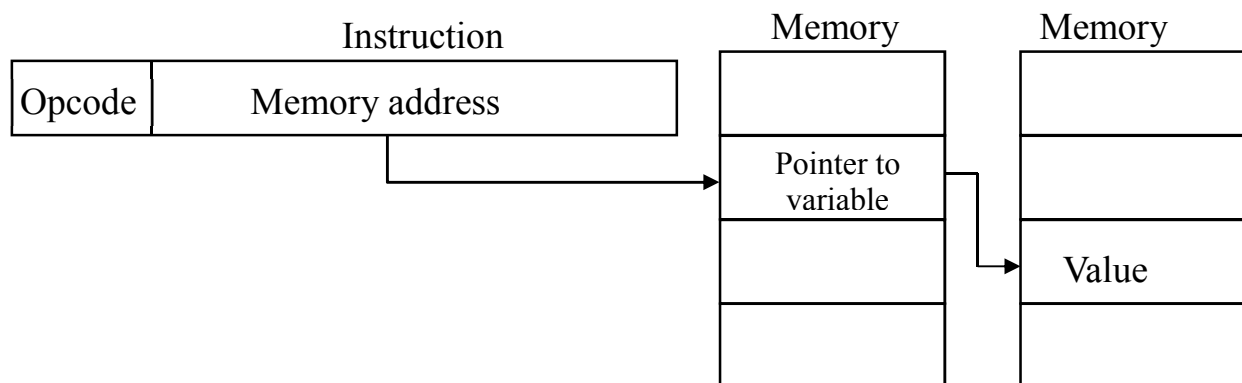
- .text – code machine of the program
- .data – initialized global variables
- .bss – uninitialized variables (initialized with 0)
- .stack – local variables, function parameters



Constants can be characterized by:

- Name
- Type (typed constants)
- Value

Pointers and indirect memory addressing mode



Pointers characteristics:

- Name – name of the variable, address of the pointer
- Type
 - o Size – size of offset addresses

- Encoding – unsigned offset address
 - Operations – operations with pointers (addresses)
- Value – address of the referred value in memory

Requirements

1. Define 3 integer and 3 float global variables as follows: 1 variable initialized with non-zero value, 1 variable initialized with zero and 1 left uninitialized, for each set. Write a C program to display for each global variable its memory address, size, encoding, alignment and value. Put in evidence also endianness when saving values to memory.
2. Define the same variables as local variables and display their memory addresses, sizes, encoding, alignment and values.
3. Define a function with three parameters and display their memory addresses, sizes, encoding, alignment and values.
4. Allocate variables in heap and display their memory addresses, sizes, encoding, alignment and values.
5. Compare memory addresses of the various sections of the program (.text, .data, .bss, .stack, heap)
6. Disassembly the executable and view the program memory sections, including variables locations.